The 8085 Master Manual

Volume VI: Handling the Unexpected

(The Interrupt Architecture)

A Grassroots Engineering Guide

Abandoning linear execution to master real-time hardware response.

"A rigid system shatters under stress; an interruptible system adapts."

Chapter 1: The Philosophy of Interruption

1.1 Polling vs. Interrupts

Up to this point, our microprocessor has operated in a strictly synchronous, linear fashion. It fetches an instruction, executes it, and moves to the next. But computers exist in a physical world where events occur asynchronously (randomly). A user presses a key, a temperature sensor trips a threshold, or a printer finishes a page.

ANALOGY: THE WAITER AND THE DOORBELL

Imagine a waiter assigned to watch the front door for customers.

METHOD 1: POLLING.

The waiter abandons all other duties, stands at the door, and stares at the handle indefinitely. In microprocessor terms, the CPU executes a continuous loop, repeatedly reading an I/O port, waiting for a status bit to change. This wastes 100% of the CPU's processing power on waiting.

METHOD 2: THE INTERRUPT.

A doorbell is installed. The waiter is free to clean tables and serve food. When a customer arrives, they press the doorbell. The waiter pauses his current task, goes to the door, handles the customer, and then returns to exactly what he was doing before. This is the hardware interrupt mechanism.

1.2 The Anatomy of a Hardware Interrupt

An **Interrupt** is an emergency hardware signal sent by an external peripheral directly to a dedicated pin on the microprocessor.

When the CPU receives a valid interrupt signal, it performs the following macroscopic sequence:

1. It completes the execution of the *current* instruction (it does not stop mid-cycle).
2. It pushes the contents of the Program Counter (the return address) onto the Stack.
3. It jumps to a specific memory location containing an **Interrupt Service Routine (ISR)**—a subroutine written explicitly to handle the emergency.
4. Once the ISR concludes (via the `RET` instruction), the CPU pops the stack and resumes its main program seamlessly.

Chapter 2: The Five Hardware Interrupt Pins

The Intel 8085 provides five distinct physical pins for hardware interrupts. Because multiple emergencies can happen simultaneously, these pins are rigidly structured in a **Priority Hierarchy**.

2.1 Classification of the Interrupts

Interrupt Pin	Priority Level	Trigger Mechanism	Maskability	Vectoring
TRAP (RST 4.5)	1 (Highest)	Edge AND Level Sensitive	Non-Maskable	Vectored
RST 7.5	2	Positive Edge Sensitive	Maskable	Vectored
RST 6.5	3	Level Sensitive	Maskable	Vectored
RST 5.5	4	Level Sensitive	Maskable	Vectored
INTR	5 (Lowest)	Level Sensitive	Maskable	Non-Vectored

2.2 Trigger Mechanisms Explained

The physical nature of the electrical signal required to trigger the CPU varies across the pins:

- **Level-Sensitive (RST 6.5, RST 5.5, INTR):** The peripheral device must hold the voltage at a logic HIGH (1) until the microprocessor finishes its current instruction and samples the pin. If the device drops the voltage to 0 before the CPU checks, the interrupt is missed.
- **Edge-Sensitive (RST 7.5):** The CPU triggers the moment it detects a transition from LOW to HIGH. The 8085 has an internal D-flip-flop dedicated to RST 7.5 that "remembers" the pulse even if the voltage drops back to zero immediately. This is ideal for brief, transient signals.
- **Edge AND Level Sensitive (TRAP):** To prevent false triggering from electrical noise, TRAP requires a transition from LOW to HIGH, *and* the signal must remain HIGH until it is acknowledged. TRAP is used for catastrophic failures, like power-loss detection, where absolute certainty is required.

Chapter 3: Vectoring Mathematics

3.1 Vectored vs. Non-Vectored Interrupts

When an interrupt is acknowledged, the CPU must jump to the Interrupt Service Routine (ISR). But how does it know the 16-bit address of the ISR?

For **Vectored Interrupts** (TRAP, RST 7.5, RST 6.5, RST 5.5), the target address is hardwired into the microprocessor's internal logic. The CPU mathematically derives the vector address and jumps there automatically without any external assistance.

For **Non-Vectored Interrupts** (INTR), the CPU does not know where to go. It must request the external hardware to place an instruction (usually an 8-bit `RST n` or a 3-byte `CALL`) onto the Data Bus via an Interrupt Acknowledge (INTA') cycle.

3.2 Deriving the Vector Addresses

THE MATHEMATICAL FORMULA FOR RST VECTORS

The vector address for any Restart (RST) interrupt is derived by multiplying the RST numerical identifier by 8 (Decimal), which is `0008H`. This grants exactly 8 bytes of memory space between each vector, providing just enough room to write a `JMP` instruction to the true ISR location.

$$\text{Vector\ Address} = \text{N imes } 0008\text{H}$$

1. VECTOR FOR RST 5.5:

5.5 imes 8 = 44 (Decimal).

Convert 44 to Hex: 44 / 16 = 2 remainder 12 (C).

VECTOR ADDRESS = 002CH

2. VECTOR FOR RST 6.5:

6.5 imes 8 = 52 (Decimal).

Convert 52 to Hex: 52 / 16 = 3 remainder 4.

VECTOR ADDRESS = 0034H

3. VECTOR FOR RST 7.5:

7.5 imes 8 = 60 (Decimal).

Convert 60 to Hex: 60 / 16 = 3 remainder 12 (C).

VECTOR ADDRESS = 003CH

4. VECTOR FOR TRAP (RST 4.5):

4.5 imes 8 = 36 (Decimal).

Convert 36 to Hex: 36 / 16 = 2 remainder 4.

VECTOR ADDRESS = 0024H

Because these addresses are mapped into Page 0 (`0000H` to `00FFH`), this section of physical memory is usually populated with ROM (Read-Only Memory) to protect the emergency interrupt vectors from being accidentally overwritten by software bugs.

Chapter 4: The Art of Masking (SIM and RIM)

4.1 Global vs. Individual Masking

There are times when a program is executing a highly critical task (e.g., precise timing loop, writing to a disk sector) where an interruption would corrupt data. The programmer must have the ability to **Mask** (ignore) external interrupts.

The 8085 has a master switch called the **Interrupt Enable (IE) Flip-Flop**.

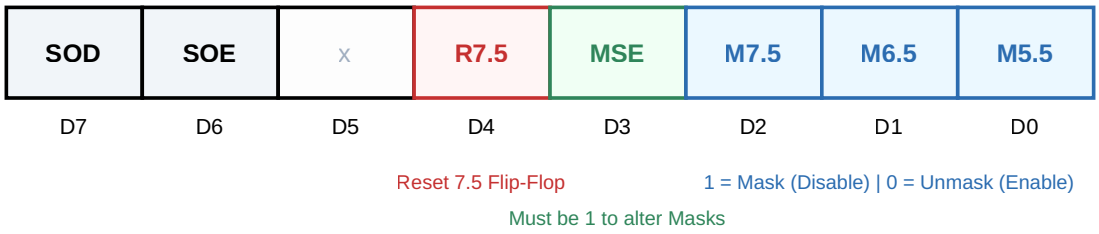
- The `DI` (Disable Interrupts) instruction resets the IE flip-flop, globally ignoring INTR, 5.5, 6.5, and 7.5.
- The `EI` (Enable Interrupts) instruction sets the IE flip-flop, globally allowing them.

However, global masking is a blunt instrument. Often, a programmer wants to ignore a low-priority keyboard interrupt (RST 5.5) but allow a high-priority sensor interrupt (RST 7.5) to pass. This requires individual manipulation of the interrupt masks using the `SIM` instruction.

4.2 The SIM Instruction (Set Interrupt Mask)

The `SIM` instruction is a 1-byte command that reconfigures the hardware mask flip-flops based entirely on the 8-bit pattern currently resting in the Accumulator.

SIM Control Word Format (Accumulator)



ALGORITHMIC EXECUTION: MASKING VIA SIM

Objective: We want to disable RST 6.5, enable RST 7.5 and RST 5.5. We also want to clear any falsely pending RST 7.5 requests. We are not using serial communication.

Bit Construction:

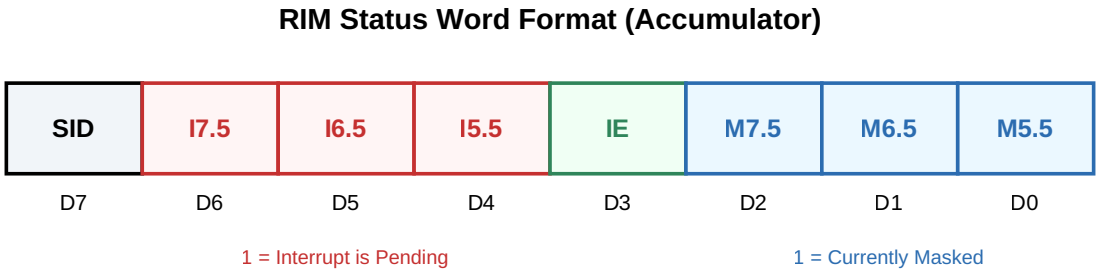
- D0 (M5.5) = 0 (Unmasked)
- D1 (M6.5) = 1 (Masked)
- D2 (M7.5) = 0 (Unmasked)
- D3 (MSE) = 1 (Master Switch ON)
- D4 (R7.5) = 1 (Clear pending flip-flop)
- D7-D5 = 000 (Ignored)

Resulting Binary: `0001 1010` = `1AH`

```
        MVI A, 1AH      ; Load Accumulator with Control Word
        SIM             ; Execute Hardware Masking
        EI              ; Globally Enable Interrupts to
finalize
```

4.3 The RIM Instruction (Read Interrupt Mask)

The `RIM` instruction does the exact opposite. It reads the internal hardware state of the 8085 and loads an 8-bit status word into the Accumulator. This is crucial for diagnostics and **Interrupt Polling**.



The true power of `RIM` lies in bits **D4, D5, and D6**. These indicate **Pending Interrupts**. If an external device sends a pulse to RST 7.5, but the processor has globally disabled interrupts (via `DI`), the processor will "remember" the pulse by setting bit D6 to 1.

A programmer can execute `RIM`, isolate bit D6 using the `ANI 40H` instruction, and determine if an emergency occurred while the CPU was busy.

Chapter 5: The Interrupt Acknowledge Cycle (INTA')

While TRAP and the RST interrupts are beautifully self-contained (Vectored), the **INTR** pin operates differently. It is Non-Vectored. When INTR is driven high, the CPU knows there is an emergency, but it does not know where the ISR is located.

THE INTR HANDSHAKE PROTOCOL

1. **THE REQUEST:**

   An external device drives the INTR pin HIGH.

2. **THE ACKNOWLEDGMENT:**

   The CPU finishes its current instruction. Instead of fetching the next sequential instruction, it generates a special machine cycle called the

   INTERRUPT ACKNOWLEDGE (INTA'**) CYCLE**

   . The CPU drives the INTA' pin LOW.

3. **HARDWARE INJECTION:**

   The external device sees INTA' go LOW. This is its cue to forcefully place an 8-bit Opcode directly onto the Data Bus. Typically, external hardware (like the 8259 Interrupt Controller) injects a `RST n` opcode (which is a 1-byte call) or a standard 3-byte `CALL` opcode.

4. **EXECUTION:**

   The CPU latches this injected opcode and executes it exactly as if it had fetched it from normal memory, jumping to the ISR.

Conclusion of the Master Manual

We have reached the culmination of our journey. Over six volumes, we have taken a microscopic rectangular die of silicon and breathed life into it.

- In **Volume I**, we built the physical buses, learning how time-division multiplexing solved the 40-pin packaging crisis.
- In **Volume II**, we explored the internal logic: the ALU, the Temp registers, the Flags, and the auto-incrementing Program Counter.
- In **Volume III**, we synchronized the system, discovering how T-States, Machine Cycles, and Wait States prevent electrical collisions.
- In **Volume IV**, we bridged the CPU to external memory and I/O using strict address decoding to prevent bus contention.
- In **Volume V**, we learned to command the machine using Assembly Language, constructing loops, subroutines, and mathematical algorithms.
- In **Volume VI**, we mastered the unpredictable, utilizing hardware interrupts to break the linear flow and respond to real-time events.

The Intel 8085 is no longer a mystery. It is a deterministic, logical machine, utterly obedient to the laws of physics and the genius of its architectural design. You possess the theoretical and mathematical foundation required to interface, program, and master it.

END OF THE MASTER MANUAL